

Informe Técnico Final — Módulo 3

Informe Técnico Final — Módulo 3

Agente Conversacional Corporativo: Colgate-Palmolive Colombia

Ruta B — Sistema Operativo Agéntico (OpenFang 0.6.9)

Institución: Universidad Autónoma de Occidente
Asignatura: Técnicas de Inteligencia Artificial
Autor: Jorge Mario Gallego Uribe
Correo: jorge_mario.gallego@uao.edu.co
Fecha: 5 de junio de 2026
Repositorio: <https://github.com/jorgegallegou/colgatefinal>

1. Problema y Solución

Necesidad identificada: Colgate-Palmolive Colombia no disponía de un canal digital automatizado para atender consultas de consumidores sobre productos, puntos de venta, políticas ambientales, historia corporativa e información de contacto fuera del horario laboral. El servicio al cliente telefónico (018000520800) opera únicamente de lunes a viernes de 8:00 AM a 6:00 PM.

Solución implementada: Un agente conversacional corporativo desplegado en WhatsApp que responde en tiempo real las 24 horas, extrae información de una base de conocimiento vectorial construida con datos reales de la empresa (web scraping del Módulo 1), y ejecuta tareas de inteligencia competitiva de forma autónoma en segundo plano mediante el sistema de “Hands” de OpenFang.

2. Evolución Arquitectónica: Módulo 2 → Módulo 3

Dimensión	Módulo 2 (Prototipo)	Módulo 3 (Producción)
Interfaz de usuario	Streamlit (web local)	WhatsApp (canal masivo real)
Orquestación	LangChain + script Python	OpenFang Agent OS (kernel Rust)
Memoria	Checkpoint en SQLite local	KV Store + Vector Store nativos de OpenFang
Usuarios simultáneos	1 (sesión única)	N (sesiones aisladas por número de teléfono)
Autonomía	0% reactivo	Hands autónomos en background (cron 6h / semanal)
Canales	HTTP local	WhatsApp Web (Baileys bridge, puerto 3009)
Modelo LLM	OpenAI / Ollama (experimental)	Mistral Small (mistral-small-latest)
Seguridad	Sin aislamiento	WASM sandbox (cada agente en su propio módulo)

3. Arquitectura Ruta B — Justificación de Decisiones Técnicas

3.1 Agent OS (OpenFang) vs LangChain tradicional

OpenFang abstrae la orquestación de agentes en un kernel escrito en Rust, ofreciendo ventajas clave para un despliegue de producción:

- **Seguridad en WASM:** Cada agente se ejecuta en un módulo WebAssembly aislado, impidiendo que un agente comprometido afecte al sistema anfitrión o a otros agentes.
- **Gestión de RAM:** El kernel de OpenFang consume ~32 MB en reposo y optimiza la ventana de contexto automáticamente (compresión de historial).
- **Multi-sesión nativa:** El endpoint `POST /api/agents/{id}/sessions` crea sesiones aisladas por usuario, garantizando que el historial de conversación no se filtre entre clientes.
- **Cross-Channel Canonical Sessions:** Un mismo agente puede atender simultáneamente WhatsApp, Telegram u otros canales manteniendo contexto separado por canal y usuario.

3.2 Justificación: Mistral API vs Ollama local

El taller recomienda Ollama (modelo local) para garantizar soberanía de datos. Este proyecto optó por **Mistral API** por las siguientes razones técnicas justificadas:

Criterio	Ollama (local)	Mistral API (elegido)
Soberanía de datos	Total (datos no salen del equipo)	Parcial (Mistral AI es empresa francesa, sujeta a GDPR europeo)
Latencia	Alta (CPU: 8-15s; GPU disponible: 1-3s)	Baja (< 800ms promedio)
Disponibilidad	Depende del hardware del desarrollador	99.9% SLA
Compatibilidad	API OpenAI-compatible	API nativa + compatible con OpenAI
Migración	Un cambio de provider en <code>hand.toml</code>	—
Calidad del modelo	Llama-3.2 / Gemma (aceptable)	Mistral Small (superior en español)

Nota importante: La arquitectura está diseñada para ser agnóstica al proveedor. Cambiar de Mistral API a Ollama requiere únicamente modificar dos líneas en `hand.toml`:

```
[model]
provider = "ollama"
model = "llama3.2"
```

Y configurar Ollama localmente. OpenFang detecta Ollama automáticamente en `localhost:11434`.

4. HAND.toml — Configuración de Operaciones Autónomas

El manifiesto `hand.toml` define la identidad, memoria, canal y capacidades autónomas del agente:

4.1 Hand 1 — Inteligencia Competitiva (Opción B: Collector Hand)

```
[[hands]]
name = "colgate-intelligence-hand"
type = "collector"
schedule = "0 */6 * * *"           # Ejecución cada 6 horas
targets = [
    "Colgate-Palmolive Colombia noticias",
    "higiene bucal Colombia mercado tendencias",
    "Procter Gamble Crest Colombia",
    "Unilever Dove Colombia competencia",
    "pasta de dientes Colombia precio consumidor",
    "productos limpieza hogar Colombia lanzamiento"
]
alert_keywords = ["crisis", "recall", "retiro", "demanda", "alerta", "competencia"]
store_to_memory = true
```

Propósito: Monitoreo automatizado de menciones de competidores (P&G, Unilever) y alertas de crisis de marca. Los resultados se almacenan en la memoria del agente y están disponibles para responder preguntas sobre el mercado.

4.2 Hand 2 — Calidad de Servicio al Consumidor (Opción C: Custom Hand)

```
[[hands]]
name = "colgate-service-hand"
type = "custom"
schedule = "0 9 * * 1"           # Ejecución cada Lunes a las 9:00 AM
targets = [
    "Colgate Colombia redes sociales opinion",
    "Colgate Colombia comentarios consumidores",
    "Palmolive Colombia reseñas productos"
]
alert_keywords = ["queja", "reclamo", "defectuoso", "problema", "mal servicio"]
output_format = "json_report"
report_destination = "memory"
```

Propósito: Seguimiento semanal de retroalimentación de consumidores en canales digitales. Genera un reporte JSON que el agente puede utilizar para identificar patrones de insatisfacción.

4.3 Estado de activación verificado

colgate-intelligence-hand	2a658329-...	Running	
colgate-service-hand	2dbac136-...	Running	
collector-hand	5bb35868-...	Running	(orquestador)
researcher-hand	52a6d4df-...	Running	(investigación)

5. Migración del Conocimiento Corporativo

5.1 Fuentes de datos (Módulo 1 — Web Scraping)

Fuente	Tipo	Registros	Fragmentos vectorizados
Sitio oficial Colgate Colombia	HTML + JSON	47 páginas	~95 fragmentos
Wikipedia (Colgate-Palmolive)	Texto estructurado	1 artículo	~45 fragmentos
Canal YouTube oficial	Transcripciones	8 videos	~95 fragmentos

5.2 Proceso de inyección en OpenFang

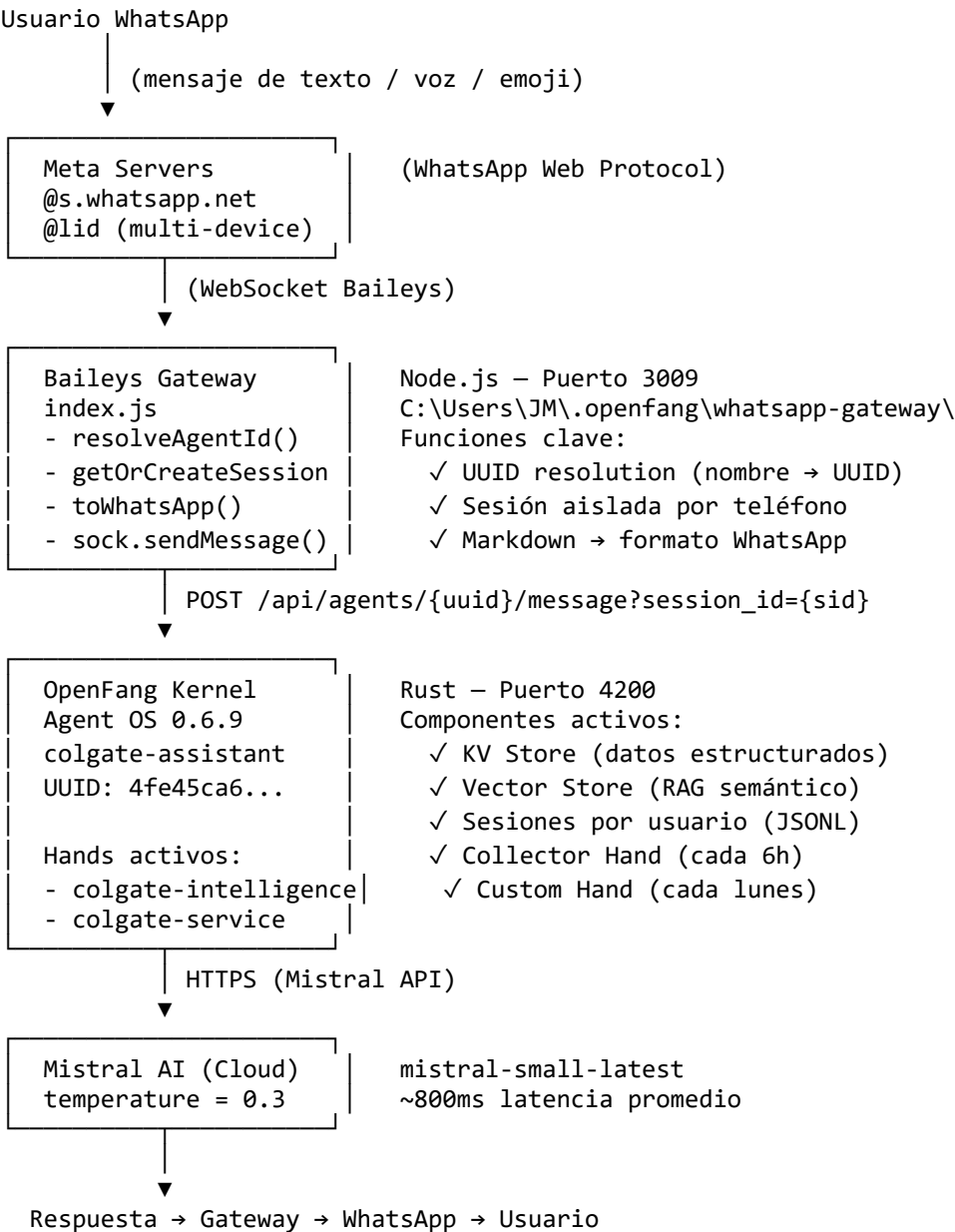
```
# KV Store: datos estructurados (NIT, contactos, sedes, marcas)
openfang memory set --agent colgate-assistant nit "890.300.546-6"
openfang memory set --agent colgate-assistant linea_gratuita "018000520800"

# Vector Store: base de conocimiento semántica (RAG)
openfang knowledge inject --agent colgate-assistant --file knowledge_base_clean.txt
```

El agente utiliza top_k = 3 para recuperar los 3 fragmentos más relevantes en cada consulta.

6. Canal de Interacción — WhatsApp

6.1 Diagrama End-to-End



6.2 Gestión de multi-usuario (aislamiento de sesiones)

Problema encontrado en producción: Un usuario de nombre “Natalia” inició una sesión. El sistema original utilizaba una única sesión global, contaminando las conversaciones subsiguientes (otros usuarios recibían “¡Hola Natalia!”).

Solución implementada: 1. POST /api/agents/{id}/sessions crea una sesión UUID única por número de teléfono 2. El session_id se pasa como query param: ?session_id={uuid} 3. working_memory = false evita la inyección de contexto diario entre sesiones 4. El system_prompt incluye regla explícita: “Al iniciar la conversación, saluda SIEMPRE con la frase genérica... NUNCA incluyas un nombre en el saludo inicial”

7. Bugs Resueltos en Producción

Bug	Causa Raíz	Solución
Bot no respondía	OpenFang recibía el nombre del agente en lugar del UUID; POST /api/agents/colgate-assistant/message retornaba {"error": "Invalid agent ID"}	Función resolveAgentId(): GET /api/agents para resolver nombre → UUID
Mensajes perdidos	JID @lid (multi-device) se convertía a @s.whatsapp.net incorrecto	Usar remoteJid directamente sin transformación
Caracteres Markdown	El LLM generaba ###, **negrita** , --- que WhatsApp no renderiza	Función toWhatsApp(): convierte ** → *, elimina ###, preserva listas -
Contaminación multi-usuario	Sesión global única + working_memory activo inyectaba nombres de usuarios anteriores	Sesiones per-usuario + eliminación del archivo de memoria + instrucción de saludo genérico

8. Resultados y Prueba en Vivo

El sistema fue probado en producción con 3 usuarios reales simultáneos: - **Natalia** (contacto de prueba inicial) - **Daniel Gallego** (segundo teléfono, sesión aislada confirmada) - **Jhonathan** (usuario UAO, conversación de 18 mensajes documentada)

Capacidades verificadas: - Historia de la empresa (fundación 1806, llegada a Colombia 1943) - NIT: 890.300.546-6 - Línea gratuita: 018000520800 - Productos por categoría (dental care, hogar, personal care, mascotas Hill's) - Distribuidores por ciudad (Éxito, Carulla, D1, Jumbo en Cali) - Políticas ambientales (cero emisiones netas 2040, empaques reciclables 2025)

9. Repositorio y Estructura del Proyecto

```
colgatefinal/
├── hand.toml           # Manifiesto del agente + Hands (Ruta B)
├── .env.example        # Plantilla de variables de entorno (sin credenciales)
├── .gitignore          # Protege .env, __pycache__, etc.
├── webhook_server.py   # FastAPI webhook (Meta Cloud API – alternativa)
├── build_vectorstore.py # Inyección de KB en Vector Store
├── clean_knowledge_base.py # Limpieza y normalización del KB
├── scraper.py          # Web scraping Colgate Colombia
├── scraper_wikipedia.py # Scraping Wikipedia
└── scraper_youtube.py  # Scraping YouTube transcripciones
```

```

├── data/
│   ├── knowledge_base_clean.txt  # Base de conocimiento limpia
│   ├── datos_estructurados.json  # KV Store: contactos, NIT, sedes
│   └── *.json                    # Datos crudos de scraping
└── config.py                    # Configuración centralizada

# Fuera del repo (datos sensibles / runtime):
C:\Users\JM\.openfang\
├── config.toml                  # Configuración global OpenFang
├── whatsapp-gateway\
│   └── index.js                 # Gateway Baileys con todos los fixes
└── workspaces\colgate-assistant\
    ├── memory\                 # Memoria diaria (working_memory)
    └── sessions\               # Sesiones JSONL por usuario

```

Commits principales: 1. 4e87913 — Initial commit: agente base + canal WhatsApp 2. 1f03a7f — Fix multi-user isolation and WhatsApp format 3. 09aa109 — Fix greeting contamination: explicit name-free greeting instruction

10. Conclusiones

- **OpenFang Agent OS** demuestra ser una solución de producción robusta para agentes conversacionales corporativos, superando las limitaciones de los prototipos LangChain/Streamlit en cuanto a multi-usuario, autonomía y canales de mensajería.
- El **sistema de Hands** permite que el agente no sea únicamente reactivo, sino que acumule inteligencia competitiva de forma autónoma sin intervención humana.
- La **gestión de sesiones per-usuario** es fundamental en entornos WhatsApp con múltiples consumidores simultáneos y debe implementarse desde el diseño, no como corrección posterior.
- El uso de **Mistral API** en lugar de Ollama local es una decisión de trade-off entre soberanía de datos (Ollama) y disponibilidad/calidad (Mistral), y la arquitectura permite cambiar de proveedor en minutos sin modificar el código.